

Deep BSDE and Tensor-Train Methods for High-Dimensional Parabolic PDEs

Reproducible benchmark and USTC 107 execution protocol

Quentinluoqz

Draft generated July 12, 2026

Abstract

Classical grid discretizations scale exponentially with spatial dimension. This project compares two alternatives on a common suite of problems with dimensions from 5 to 100: Deep BSDE solvers, which exploit stochastic representations and neural approximation, and tensor-train methods, which exploit low-rank structure. The experimental protocol jointly measures solution error, uncertainty, wall-clock cost, memory, numerical precision, and hardware throughput. At the current local-development gate, all scientific interfaces, reference solvers, resumable training, tensor-train representation experiments, and Slurm orchestration are implemented. Quantitative conclusions are intentionally deferred until the USTC 107 campaign artifacts have been returned and provenance-validated.

1 Motivation and research questions

For a tensor-product grid with n points per coordinate, a direct representation requires n^d values. At $d = 100$ this is intractable even for very small n . Deep BSDE methods replace the grid by Monte Carlo paths and trainable gradient approximations. Tensor trains replace a dense tensor by a chain of low-rank cores. Both can avoid an explicit exponential representation, but they make different structural and optimization assumptions.

The benchmark addresses four primary questions: how error scales with dimension; how the cost–accuracy Pareto frontier changes; when low TT rank is retained; and when fp32 or TF32 is sufficient relative to fp64. Seed variance and failure modes are retained as scientific evidence rather than silently removed.

2 Problem class and stochastic representation

We consider semilinear terminal-value problems

$$\partial_t u + \mu \cdot \nabla u + \frac{1}{2} \text{Tr}(\sigma \sigma^\top \nabla^2 u) + f(t, x, u, \sigma^\top \nabla u) = 0, \quad u(T, x) = g(x). \quad (1)$$

With X following the forward SDE induced by μ and σ , the associated backward equation evolves $Y_t = u(t, X_t)$ and $Z_t = \sigma^\top \nabla u(t, X_t)$. Deep BSDE optimizes the terminal mismatch $\mathbb{E}|Y_T - g(X_T)|^2$.

2.1 Benchmark equations

For E3 the Cole–Hopf transform gives

$$u(t, x) = -\lambda^{-1} \log \mathbb{E} \left[\exp \{ -\lambda g(x + \sqrt{2} W_{T-t}) \} \right]. \quad (2)$$

ID	Type	Reference	Default horizon
E1	Black–Scholes–Barenblatt	analytic quadratic solution	$T = 1$
E2	linear-quadratic control	scalar Riccati solution	$T = 1$
E3	quadratic-gradient HJB	Cole–Hopf Monte Carlo, 95% CI	$T = 1$
E4	Allen–Cahn	literature value at $d = 100, x = 0$ only	$T = 0.3$
E5	Gaussian heat equation	analytic heat kernel	$T = 1$

Table 1: The reference boundary is part of the data model. E4 errors outside the canonical point are undefined, not estimated.

The implementation samples in bounded-memory batches and propagates the standard error through the logarithm to form a 95% confidence interval. The production reference uses 10^7 samples. Tests compare the one-dimensional case with deterministic quadrature and verify the expected square-root sample-size behavior of the interval width.

3 Deep BSDE methods

M1A learns $u(0, x_0)$, an initial Z_0 , and an independent network for every internal time step. Each step network contains two linear–batch-normalization–ReLU blocks. M1B shares a tanh network with input (t, x) across all steps. Both use the same Euler–Maruyama paths, optimizer lifecycle, result schema, and failure policy.

Checkpoints contain the model, optimizer, scheduler, iteration, loss history, complete configuration hash, and Python, NumPy, Torch, and CUDA random-number states. Resume rejects a mismatched configuration. Non-finite losses, gradients, or parameters terminate with a **diverged** metric rather than disappearing from the campaign.

The precision modes are deliberately narrow: fp32 uses float32; TF32 keeps float32 tensors and enables CUDA tensor-core matrix multiplication; fp64 converts parameters, paths, and equation operations to double. AMP and half precision are outside this comparison.

4 Tensor-train methods

Tier 1 measures the compressibility of known solution functions. A rank-adaptive TT-cross algorithm samples a 32-point Chebyshev grid per dimension without materializing the full tensor. It uses held-out validation and test indices, records the achieved rank spectrum, compression, function calls, wall-clock time, and both discrete and continuous interpolation errors. Failure to reach a target tolerance is recorded as **not_reached**.

Tier 2 is a conditional external integration. The Richter–Sallandt–Nüsken solver relies on xerus/SALSA, which has compiled dependencies and AGPL obligations. No external source is vendored. A pinned adapter is enabled only after a recorded license and no-sudo build review; otherwise Tier 1 remains the reproducible tensor baseline.

5 Experimental design

The mandatory campaign contains 120 Deep BSDE runs, 45 rank experiments, and 36 precision runs. Optional accepted Tier-2 integration adds 30 runs. The strong-scaling experiment uses a fixed global batch for one versus two A100 GPUs. Manifests are immutable, one row is one atomic command, and each row binds method, equation, dimension, seed, precision, source commit, and config hash.

Campaign	Planned runs	Primary resource	Gate
Deep BSDE main	120	RTX 5090	pilot E1 then E3
TT rank	45	CPU-6530	E5 low-rank smoke
Precision	36	RTX 5090 / A100	matched configs
Strong scaling	6	A100	fixed global batch
TT Tier 2	30	CPU	license and build review

Table 2: Planned campaign sizes. The manifest generator is the executable source of truth.

6 Platform execution and evidence

Static partition and QoS values from the training material are treated as defaults only. Every campaign starts with live `sinfo`, `scontrol`, `sacctmgr`, module, Python, Torch, CUDA, and storage snapshots. Login nodes perform only preparation and job management. GPU tests execute inside allocated jobs.

RTX 5090 arrays default to one GPU and four CPU cores with concurrency four; A100 arrays default to concurrency two. Templates use one task and explicit CPUs per task. Checkpointing, signal handling, and requeue preserve interrupted work. `sacct` rows, stdout/stderr, Grafana screenshots, and environment snapshots are returned with the atomic metrics. Slurm JWTs remain user-controlled environment variables and are never written to source, logs, bundles, or chat.

7 Results protocol

The local repository currently contains 0 integrated result rows, 0 completed rows, and 0 non-completed rows from 0 source commits. These zero-valued draft macros are replaced directly from the validated summary table after cloud return; they are not hand-edited.

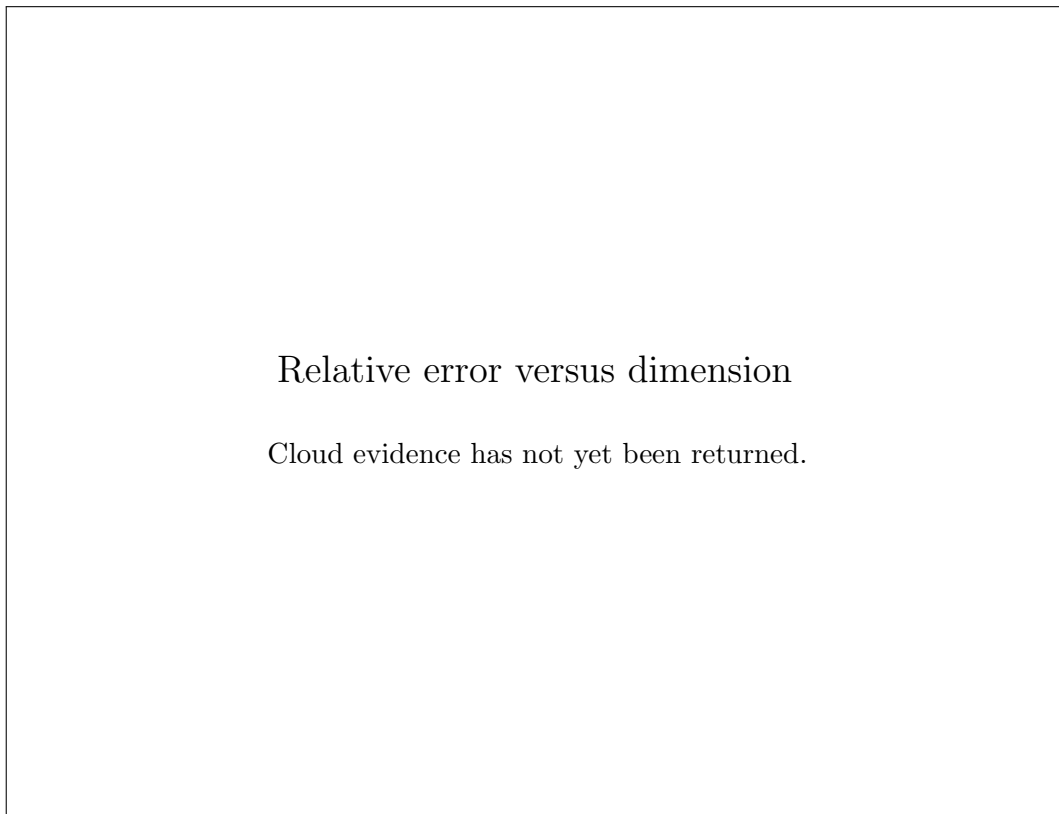


Figure 1: Relative error versus dimension. Missing panels remain explicit rather than being inferred.

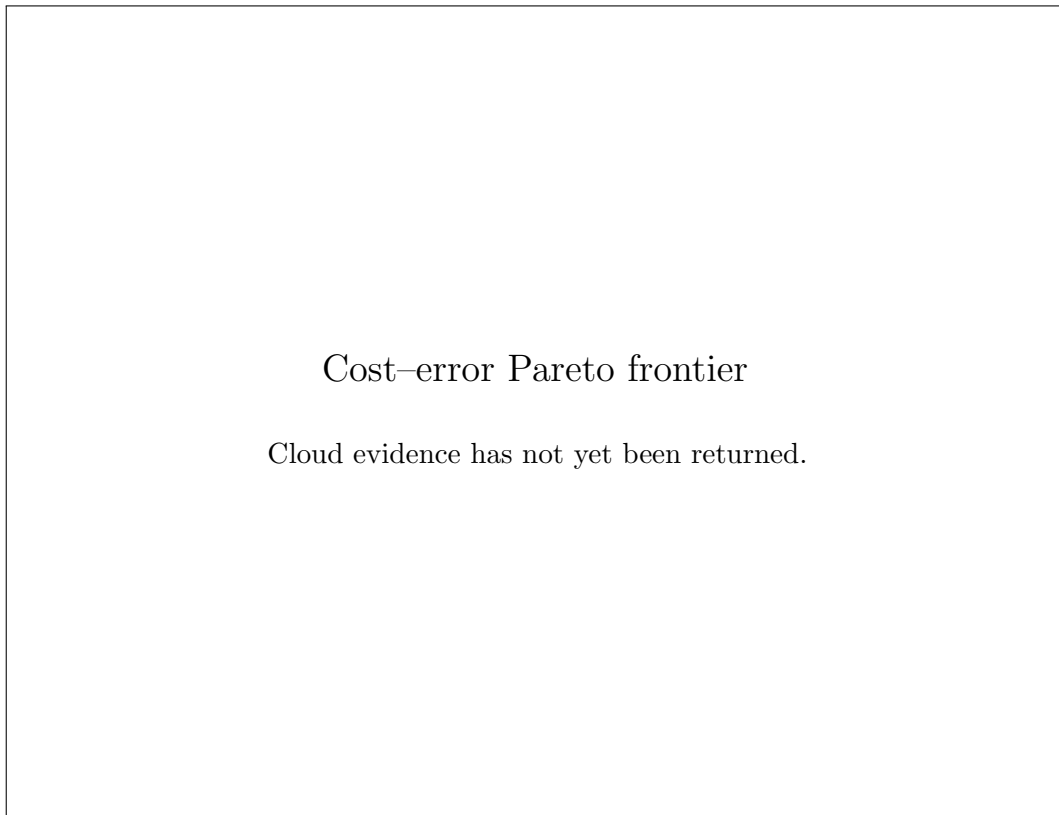


Figure 2: Cost-error Pareto frontier. Missing panels remain explicit rather than being inferred.

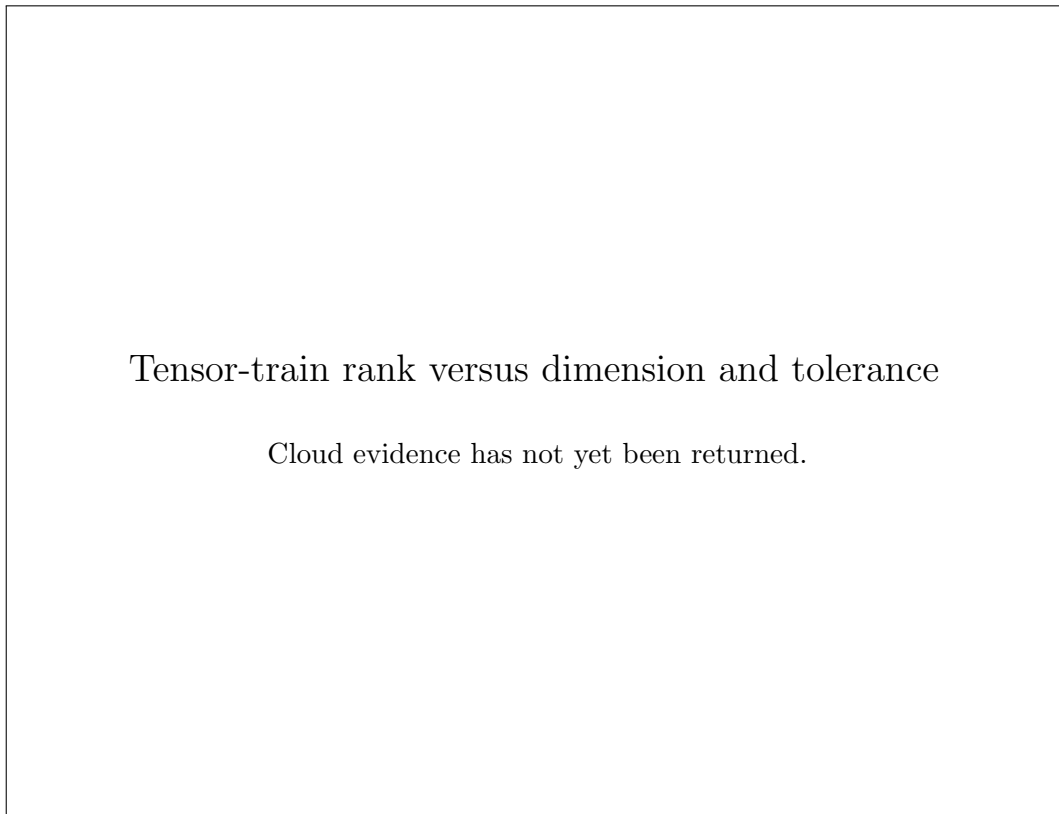


Figure 3: Tensor-train rank versus dimension and tolerance. Missing panels remain explicit rather than being inferred.

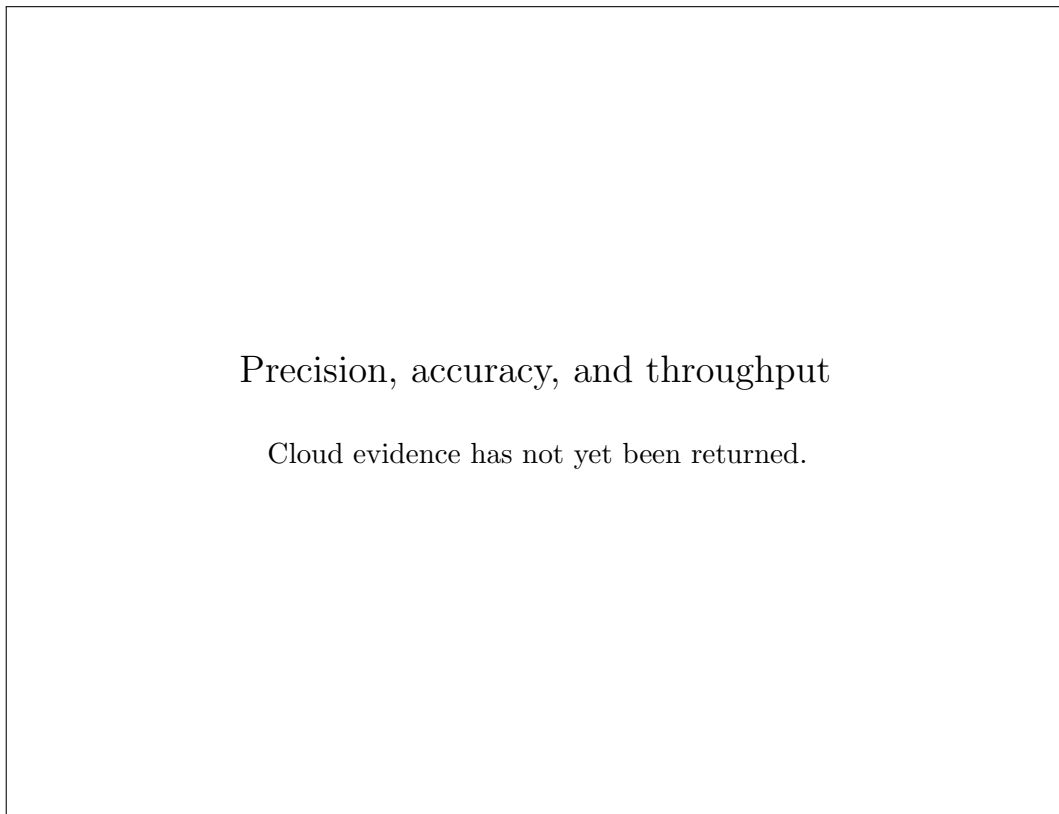


Figure 4: Precision, accuracy, and throughput. Missing panels remain explicit rather than being inferred.

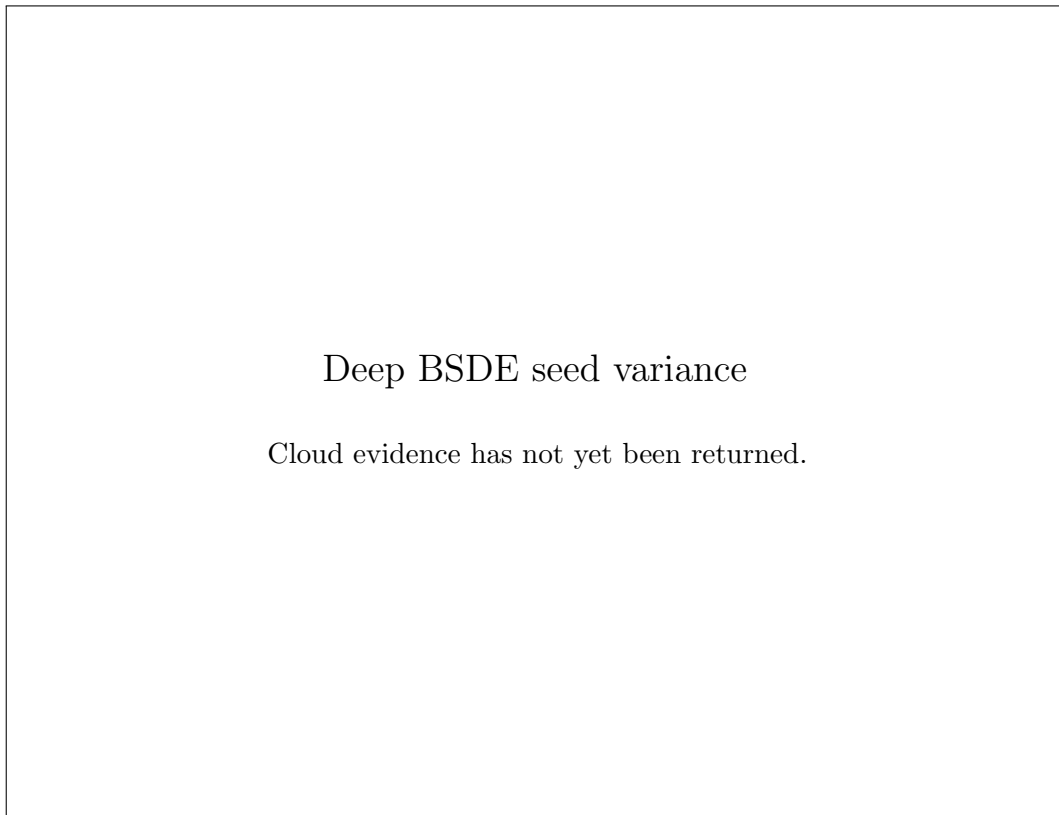


Figure 5: Deep BSDE seed variance. Missing panels remain explicit rather than being inferred.

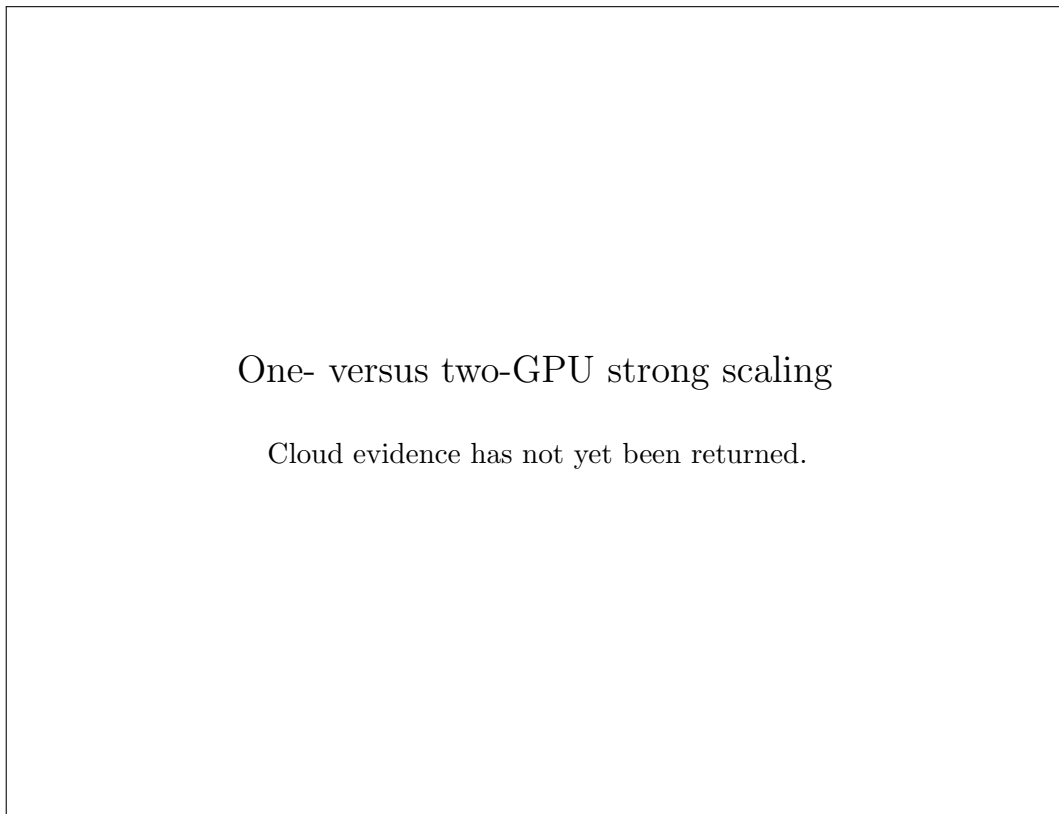


Figure 6: One- versus two-GPU strong scaling. Missing panels remain explicit rather than being inferred.

No numerical conclusion is stated until each plotted point has a valid reference boundary, source commit, configuration hash, and accounting record. Incomplete evidence produces a visible missing panel or warning instead of an interpolated claim.

8 Validation and reproducibility

Fast tests cover configuration rejection, registry behavior, analytic references, batched tensor shapes in dimensions 1, 5, and 50, Monte Carlo confidence intervals, checkpoint continuity, manifest expansion, array indexing, and graceful incomplete plotting. GPU and long numerical tests are explicit markers. A clean clone must pass format, lint, type, and CPU test gates before a cloud bundle can be created.

9 Limitations and planned completion

The current report is a local-stage artifact. It does not claim the target E1 or E3 cloud accuracy, hardware throughput, scaling efficiency, or a successful Tier-2 build. Those claims require the user-run USTC campaign and returned evidence. Hyperparameter conclusions will be restricted to the frozen campaign rather than generalized beyond the tested range.

10 Conclusion

The project provides a common scientific and engineering boundary for two distinct ways of addressing high dimensionality. Its principal design choice is that numerical uncertainty, failure, resource use, and provenance are first-class outputs. The final comparison will be written only after the cloud artifacts pass the same boundary.

A Atomic command contract

```
python scripts/run_one.py --method M --equation E --dim D \  
    --seed S --precision P [--resume] [--override key=value]
```

B Required repository gates

```
uv run ruff format --check src tests experiments scripts  
uv run ruff check src tests experiments scripts  
uv run mypy src  
uv run pytest -m "not slow and not gpu"  
uv run pre-commit run --all-files
```

References

- [1] W. E, J. Han, and A. Jentzen. Deep learning-based numerical methods for high-dimensional parabolic PDEs and BSDEs. *Communications in Mathematics and Statistics*, 5:349–380, 2017.
- [2] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *PNAS*, 115:8505–8510, 2018.
- [3] L. Richter, L. Sallandt, and N. Nüsken. Solving high-dimensional parabolic PDEs using the tensor train format. *PMLR*, 139:8998–9009, 2021.